

Docker Compose 샌드박스 20가지 개발 격리 검증 교본

Docker Compose 샌드박스 20가지 개발 및 격리 검증 실전 교본

Codex CLI · SSH · MariaDB · Kotlin/Spring Boot · Rust · Redis · Ollama · Kafka/Redpanda · Traefik · Prometheus

본 문서는 개발 및 격리 검증을 위한 20가지 Docker Compose 샌드박스 구성을 체계적으로 정리한 실전 자료임. 각 구성은 특정 아키텍처나 검증 목적을 갖도록 설계하며, 서비스 간 통신은 Docker 내부 네트워크를 기본으로 하고 필요한 포트만 호스트에 공개하는 방식을 사용함.

이 문서에서 고친 곳

- root 비밀번호만 정하고 PermitRootLogin 을 바꾸지 않던 예제 4곳(LLM·TUI·크로스 컴파일·관리 콘솔)에 sed 한 줄을 더했습니다. 기본값(prohibit-password)에서는 root 비밀번호 SSH 로그인인 거부됩니다. 다른 예제들이 이미 쓰던 줄과 같습니다.
- 문서 생성 과정의 대화 흔적(「사용자가 제공한」)을 지웠습니다.

1. 문서 사용 전 중요 보안 및 정확성 안내

- 아래 예제의 비밀번호는 학습용 예시임. 실제 환경에서는 .env, Docker secrets 또는 별도 Secret Manager를 사용함.
- SSH에서 root 비밀번호 로그인을 허용하는 예제는 격리된 로컬 개발 검증용으로만 사용함. 장기 사용 시 일반 사용자 + SSH public key 방식으로 변경하는 것이 바람직함.
- OpenAI API 키를 docker-compose.yml에 직접 적지 않고 `\${OPENAI\_API\_KEY}`처럼 환경변수로 주입함.
- Codex CLI 설치 명령은 설치 시점의 공식 패키지명과 CLI 버전에 따라 달라질 수 있으므로 실제 구축 시 공식 OpenAI 문서의 현재 설치 명령을 확인함.
- MariaDB master/slave 예제는 컨테이너 두 개를 배치한 골격임. 실제 복제 검증에는 server-id, binlog, replication user, CHANGE MASTER/CHANGE REPLICATION SOURCE 등의 추가 초기화가 필요함.
- Docker Compose 최신 구현에서는 최상위 `version:` 필드가 필수가 아님. 아래 예제에도 version: 필드는 넣지 않았음.

2. 20가지 구성 한눈에 보기

No.	구성	핵심 조합
1	올인원 통합 개발	Codex/SSH + Kotlin + Rust + MariaDB
2	보안 원격 Rust 개발	SSH + Rust + Codex
3	DB 복제 검증	Spring Boot + MariaDB Master/Replica
4	고성능 비동기	Rust Axum/Tokio + MariaDB + Redis
5	로컬 LLM	Ollama + Codex + SSH
6	Polyglot RPC	Spring WebFlux + Rust gRPC + MariaDB
7	CI/CD 테스트	Kotlin Gradle + Rust Cargo + 임시 DB
8	Bastion 격리	SSH Bastion + 내부 DB/API
9	시계열/연산	Rust + MariaDB + Redis Pub/Sub

No.	구성	핵심 조합
10	TUI 관리	Rust Ratatui + Spring Admin
11	포렌식	Rust parser + Kotlin reporter + MariaDB
12	이벤트 메시징	Redpanda/Kafka + Rust + Kotlin
13	ARM 크로스빌드	aarch64 Rust + SSH
14	API Gateway	Traefik + Kotlin + Rust
15	Hardened	Non-root + cap_drop + resource limits
16	멀티테넌트	2 MariaDB + Kotlin routing
17	Job Queue	Redis + Kotlin producer + Rust worker
18	WebSocket	Rust WebSocket + Kotlin simulator
19	Cloud IDE	Code-Server + Codex + Rust + JDK
20	프로덕션 모사	MariaDB + Redis + SSH/Codex + Spring + Rust + Prometheus

섹션 1. 올인원 통합 개발 샌드박스

SSH 터미널, Codex CLI, Kotlin/Spring Boot, Rust 워커, MariaDB가 하나의 bridge 네트워크에서 연결되는 표준 풀스택 개발 환경임. MariaDB는 named volume으로 영구 저장하고 Spring Boot와 Rust는 서비스명 `mariadb`로 DB에 접근함.

핵심 포인트: 표준 학습용 풀스택 · DB 데이터 영구 보존 · Codex 작업 폴더 host mount · 8080/2222/3306 외부 접속

Docker Compose 예제

yaml

```
services:
  mariadb:
    image: mariadb:11.2
    container_name: sbx-mariadb
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: app_sandbox
      MYSQL_USER: devuser
      MYSQL_PASSWORD: ${DB_APP_PASSWORD}
    ports:
      - "3306:3306"
    volumes:
      - mariadb_data:/var/lib/mysql
    networks: [sandbox-net]
    healthcheck:
      test: ["CMD", "mariadb-admin", "ping", "-h", "localhost", "-u", "root", "-p${DB_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 10

  ssh-codex-dev:
    image: ubuntu:24.04
    container_name: sbx-ssh-codex
    restart: unless-stopped
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server curl git build-essential
      pkg-config libssl-dev nodejs npm &&
      mkdir -p /var/run/sshd &&
      echo 'root:${SSH_DEV_PASSWORD}' | chpasswd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/'
      /etc/ssh/sshd_config &&
      npm install -g @openai/codex &&
      /usr/sbin/sshd -D"
    ports:
      - "2222:22"
    environment:
      OPENAI_API_KEY: ${OPENAI_API_KEY}
    volumes:
      - ./workspace:/workspace
    working_dir: /workspace
    networks: [sandbox-net]

  kotlin-springboot:
    image: eclipse-temurin:21-jdk-jammy
    container_name: sbx-springboot
    working_dir: /app
    volumes:
      - ./backend-kotlin:/app
    environment:
      SPRING_DATASOURCE_URL: jdbc:mariadb://mariadb:3306/app_sandbox
      SPRING_DATASOURCE_USERNAME: devuser
      SPRING_DATASOURCE_PASSWORD: ${DB_APP_PASSWORD}
    command: sh -c "./gradlew bootRun"
```

```

ports:
  - "8080:8080"
depends_on:
  mariadb:
    condition: service_healthy
networks: [sandbox-net]

rust-worker:
  image: rust:1.80-bullseye
  container_name: sbx-rust-worker
  working_dir: /app
  volumes:
    - ./worker-rust:/app
  environment:
    DATABASE_URL: mysql://devuser:${DB_APP_PASSWORD}@mariadb:3306/app_sandbox
  command: bash -c "cargo install cargo-watch && cargo watch -x run"
  depends_on:
    mariadb:
      condition: service_healthy
  networks: [sandbox-net]

volumes:
  mariadb_data:

networks:
  sandbox-net:
    driver: bridge

```

구동 및 확인

해당 예제를 `docker-compose.yml` 로 저장한 후 다음 순서로 검증함.

```

docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100

```

bash

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 2. 보안 원격 개발 컨테이너 (SSH + Rust + Codex)

Rust 공식 이미지를 기반으로 SSH와 Codex를 추가하여 원격 터미널 중심으로 개발하는 구성임. Cargo cache를 named volume으로 유지하여 반복 빌드 시간을 줄이는 목적임.

핵심 포인트: Rust 원격 개발 · Cargo cache 영구화 · SSH 2201 · Codex CLI

Docker Compose 예제

```
services:
  remote-codex-rust:
    image: rust:1.80-slim-bookworm
    container_name: sbx-remote-rust-cli
    environment:
      OPENAI_API_KEY: ${OPENAI_API_KEY}
    ports:
      - "2201:22"
    volumes:
      - ./rust_project:/root/project
      - rust_cargo_cache:/usr/local/cargo
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server nodejs npm git zsh curl &&
      echo 'root:${SSH_RUST_PASSWORD}' | chpasswd &&
      mkdir -p /var/run/sshd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/'
      /etc/ssh/sshd_config &&
      npm install -g @openai/codex &&
      /usr/sbin/sshd -D"
    networks: [isolated-dev]

volumes:
  rust_cargo_cache:

networks:
  isolated-dev:
    driver: bridge
```

yaml

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

bash

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

섹션 3. Kotlin/Spring Boot & MariaDB 레플리카 테스트 환경

Spring Boot에서 쓰기 DB와 읽기 DB를 분리하는 애플리케이션 구조를 검증하기 위한 골격임. 단, 컨테이너 두 개를 띄우는 것만으로 MariaDB 복제가 자동 구성되지는 않으므로 실제 replication 설정 파일과 초기화 SQL이 추가로 필요함.

핵심 포인트: 다중 datasource · 읽기/쓰기 라우팅 검증 · 실제 replication 초기화는 별도

Docker Compose 예제

yaml

```
services:
  mariadb-master:
    image: mariadb:11.2
    container_name: sbx-mariadb-master
    environment:
      MYSQL_ROOT_PASSWORD: ${MASTER_ROOT_PASSWORD}
      MYSQL_DATABASE: replication_db
      MYSQL_USER: repl_user
      MYSQL_PASSWORD: ${REPL_PASSWORD}
    ports:
      - "3306:3306"
    volumes:
      - db_master_data:/var/lib/mysql
    networks: [repl-net]

  mariadb-slave:
    image: mariadb:11.2
    container_name: sbx-mariadb-slave
    environment:
      MYSQL_ROOT_PASSWORD: ${SLAVE_ROOT_PASSWORD}
    ports:
      - "3307:3306"
    volumes:
      - db_slave_data:/var/lib/mysql
    networks: [repl-net]

  kotlin-app:
    image: gradle:8.7-jdk21
    container_name: sbx-kotlin-multi-ds
    working_dir: /app
    volumes:
      - ./app-src:/app
    environment:
      SPRING_DATASOURCE_WRITE_URL: jdbc:mariadb://mariadb-master:3306/replication_db
      SPRING_DATASOURCE_READ_URL: jdbc:mariadb://mariadb-slave:3306/replication_db
      DB_USER: repl_user
      DB_PASSWORD: ${REPL_PASSWORD}
    command: gradle bootRun
    ports:
      - "8080:8080"
    networks: [repl-net]

volumes:
  db_master_data:
  db_slave_data:

networks:
  repl-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

bash

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 4. 고성능 Rust 비동기 백엔드 & MariaDB 풀링

Axum/Tokio 서비스에 MariaDB와 Redis를 결합하여 DB connection pool, 비동기 처리, cache hit/miss를 실험하는 구성임.

핵심 포인트: Axum/Tokio · Redis cache · MariaDB · Rust release 실행

Docker Compose 예제

```
services:
  mariadb:
    image: mariadb:10.11
    container_name: sbx-perf-db
    environment:
      MYSQL_ROOT_PASSWORD: ${PERF_DB_PASSWORD}
      MYSQL_DATABASE: async_perf
    ports:
      - "3308:3306"
    volumes:
      - perf_data:/var/lib/mysql
    networks: [perf-net]

  redis:
    image: redis:7-alpine
    container_name: sbx-perf-redis
    ports:
      - "6379:6379"
    networks: [perf-net]

  rust-axum:
    image: rust:1.80-bullseye
    container_name: sbx-rust-axum
    working_dir: /service
    volumes:
      - ./rust_axum_service:/service
    environment:
      DATABASE_URL: mysql://root:${PERF_DB_PASSWORD}@mariadb:3306/async_perf
      REDIS_URL: redis://redis:6379
    command: bash -c "cargo run --release"
    ports:
      - "9000:9000"
    depends_on: [mariadb, redis]
    networks: [perf-net]

volumes:
  perf_data:

networks:
  perf-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 5. 로컬 LLM(Ollama) + Codex CLI + SSH 협업 환경

Ollama 모델 저장소를 named volume으로 유지하고 SSH 개발 노드와 같은 내부 네트워크에 배치하는 실험형 구성임.
Codex 자체의 모델 공급자/endpoint 호환성은 사용하는 Codex 버전과 설정을 별도로 확인해야 함.

핵심 포인트: Ollama 11434 · Codex/SSH 개발노드 · 모델 volume 영구화

Docker Compose 예제

```
services:
  ollama-engine:
    image: ollama/ollama:latest
    container_name: sbx-ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama_models:/root/.ollama
    networks: [llm-net]

  ssh-codex:
    image: debian:12-slim
    container_name: sbx-ssh-llm-cli
    environment:
      OPENAI_API_KEY: ${OPENAI_API_KEY}
      OLLAMA_URL: http://ollama-engine:11434
    ports:
      - "2223:22"
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server git curl nodejs npm &&
      echo 'root:${SSH_LLM_PASSWORD}' | chpasswd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config
&&
      mkdir -p /var/run/sshd &&
      npm install -g @openai/codex &&
      /usr/sbin/sshd -D"
    depends_on: [ollama-engine]
    networks: [llm-net]

volumes:
  ollama_models:

networks:
  llm-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

섹션 6. Polyglot Microservices: Spring Boot Gateway + Rust gRPC

Spring WebFlux 계층과 Rust gRPC 엔진을 분리하여 언어 간 RPC 및 장애 전파를 검증함.

핵심 포인트: gRPC · WebFlux · Polyglot

Docker Compose 예제

```
services:
  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${POLYGLOT_DB_PASSWORD}
      MYSQL_DATABASE: polyglot_db
    networks: [internal-rpc]

  rust-grpc-core:
    image: rust:1.80-bullseye
    working_dir: /app
    volumes: [./rust_grpc:/app]
    environment:
      DATABASE_URL: mysql://root:${POLYGLOT_DB_PASSWORD}@mariadb:3306/polyglot_db
    command: cargo run --bin server
    ports: ["50051:50051"]
    depends_on: [mariadb]
    networks: [internal-rpc]

  kotlin-gateway:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /gateway
    volumes: [./gateway-src:/gateway]
    environment:
      GRPC_SERVER_HOST: rust-grpc-core
      GRPC_SERVER_PORT: "50051"
    command: ./gradlew bootRun
    ports: ["8080:8080"]
    depends_on: [rust-grpc-core]
    networks: [internal-rpc]

networks:
  internal-rpc:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 7. 자동화 CI/CD & 테스트 러너

Kotlin Gradle check와 Rust cargo test를 격리 컨테이너에서 수행하고 DB는 tmpfs로 휘발시킴.

핵심 포인트: CI 시뮬레이션 · tmpfs DB · Kotlin/Rust 병렬 검증

Docker Compose 예제

```
services:
  ephemeral-mariadb:
    image: mariadb:10.11
    environment:
      MYSQL_ALLOW_EMPTY_PASSWORD: "yes"
      MYSQL_DATABASE: ci_test
    tmpfs: [/var/lib/mysql]
    networks: [ci-net]

  test-runner-kotlin:
    image: gradle:8.7-jdk21
    working_dir: /ci
    volumes: [./kotlin-project:/ci]
    environment:
      TEST_DB_HOST: ephemeral-mariadb
    command: gradle check --info
    depends_on: [ephemeral-mariadb]
    networks: [ci-net]

  test-runner-rust:
    image: rust:1.80
    working_dir: /ci
    volumes: [./rust-project:/ci]
    environment:
      DATABASE_URL: mysql://root@ephemeral-mariadb:3306/ci_test
    command: cargo test --verbose
    depends_on: [ephemeral-mariadb]
    networks: [ci-net]

networks:
  ci-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

섹션 8. Bastion Host 기반 보안 서브넷

외부에서는 Bastion만 접근하고 DB와 API는 internal network에 두어 직접 노출을 막는 구성임.

핵심 포인트: Bastion · internal network · 최소 포트 공개

Docker Compose 예제

```
services:
  ssh-bastion:
    image: linuxserver/openssh-server:latest
    environment:
      PUID: "1000"
      PGID: "1000"
      PASSWORD_ACCESS: "true"
      USER_PASSWORD: ${BASTION_PASSWORD}
      USER_NAME: developer
    ports: ["2224:2222"]
    networks: [dmz-net, secure-backend]

  mariadb-internal:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${INTERNAL_DB_PASSWORD}
      MYSQL_DATABASE: internal_core
    networks: [secure-backend]

  kotlin-backend:
    image: eclipse-temurin:21-jre-alpine
    environment:
      DB_HOST: mariadb-internal
    networks: [secure-backend]

networks:
  dmz-net:
    driver: bridge
  secure-backend:
    internal: true
```

yaml

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

bash

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

섹션 9. Rust 고빈도 연산 & 시계열 저장

Rust 수집/연산 엔진과 Redis Pub/Sub, MariaDB 저장 계층을 조합함.

핵심 포인트: 고빈도 연산 · Redis Pub/Sub · MariaDB tuning

Docker Compose 예제

```
services:
  mariadb-timeseries:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${TS_DB_PASSWORD}
      MYSQL_DATABASE: metrics_ts
    ports: ["3309:3306"]
    command: --innodb-buffer-pool-size=512M --innodb-flush-log-at-trx-commit=2
    networks: [quant-net]

  redis-feed:
    image: redis:7-alpine
    ports: ["6380:6379"]
    networks: [quant-net]

  rust-engine:
    image: rust:1.80-bullseye
    working_dir: /quant
    volumes: [./rust_quant:/quant]
    environment:
      DB_URL: mysql://root:${TS_DB_PASSWORD}@mariadb-timeseries:3306/metrics_ts
      REDIS_URL: redis://redis-feed:6379
    command: cargo run --release --bin market_collector
    networks: [quant-net]

networks:
  quant-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 10. SSH TUI 관리자 & Spring Boot Admin

Rust Ratatui CLI 관리 도구와 Spring 관리 API를 같은 DB에 연결하는 운영도구 실험 환경임.

핵심 포인트: Ratatui · Spring Admin · SSH

Docker Compose 예제

```
services:
  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${TUI_DB_PASSWORD}
      MYSQL_DATABASE: manage_center
    networks: [tui-net]

  admin-spring:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /srv
    volumes: [./spring-admin:/srv]
    environment:
      SPRING_DATASOURCE_URL: jdbc:mariadb://mariadb:3306/manage_center
    command: ./gradlew bootRun
    ports: ["8081:8080"]
    networks: [tui-net]

  ssh-rust-tui:
    image: rust:1.80-bullseye
    ports: ["2225:22"]
    volumes: [./rust-tui:/opt/tui]
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server cmake pkg-config libncurses5-dev &&
      echo 'root:${TUI_SSH_PASSWORD}' | chpasswd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config
&&
      mkdir -p /var/run/sshd &&
      /usr/sbin/sshd -D"
    networks: [tui-net]

networks:
  tui-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 11. 데이터 포렌식 및 바이너리 파싱

증거 샘플은 read-only로 mount하고 Rust parser가 분석 메타데이터를 MariaDB에 저장하며 Kotlin reporter가 결과를 생성함.

핵심 포인트: read-only evidence · Rust parser · Kotlin reporter

Docker Compose 예제

```
services:
  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${FORENSICS_DB_PASSWORD}
      MYSQL_DATABASE: forensics_evidence
    volumes: [forensic_db_vol:/var/lib/mysql]
    networks: [parse-net]

  rust-parser:
    image: rust:1.80-bullseye
    working_dir: /analysis
    volumes:
      - ./evidence_samples:/samples:ro
      - ./rust_parser:/analysis
    environment:
      DB_URL: mysql://root:${FORENSICS_DB_PASSWORD}@mariadb:3306/forensics_evidence
    command: cargo run --bin raw_disk_parser
    networks: [parse-net]

  kotlin-reporter:
    image: eclipse-temurin:21-jre
    working_dir: /reporter
    volumes: [./reporter_build:/reporter]
    environment:
      DB_HOST: mariadb
    command: java -jar reporter.jar
    networks: [parse-net]

volumes:
  forensic_db_vol:

networks:
  parse-net:
    driver: bridge
```

yaml

구동 및 확인

해당 예제를 `docker-compose.yml` 로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

bash

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 12. 이벤트 기반 메시징 (Redpanda/Kafka)

Kafka 호환 Redpanda를 중심으로 Rust producer와 Kotlin consumer를 연결하는 이벤트 기반 검증 환경임.

핵심 포인트: Kafka 호환 · producer/consumer · event store

Docker Compose 예제

yaml

```
services:
  redpanda:
    image: docker.redpanda.com/redpandadata/redpanda:v24.1.1
    command:
      - redpanda
      - start
      - --smp
      - "1"
      - --memory
      - 512M
      - --overprovisioned
      - --kafka-addr
      - internal://0.0.0.0:9092,external://0.0.0.0:19092
      - --advertise-kafka-addr
      - internal://redpanda:9092,external://localhost:19092
    ports: ["19092:19092"]
    networks: [event-net]

  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${EVENT_DB_PASSWORD}
      MYSQL_DATABASE: event_store
    networks: [event-net]

  rust-producer:
    image: rust:1.80-slim
    working_dir: /producer
    volumes: [./rust_producer:/producer]
    environment:
      KAFKA_BROKERS: redpanda:9092
    command: cargo run
    depends_on: [redpanda]
    networks: [event-net]

  kotlin-consumer:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /consumer
    volumes: [./kotlin_consumer:/consumer]
    environment:
      KAFKA_SERVER: redpanda:9092
      SPRING_DATASOURCE_URL: jdbc:mariadb://mariadb:3306/event_store
    command: ./gradlew bootRun
    depends_on: [redpanda, mariadb]
    networks: [event-net]

networks:
  event-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

bash

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 13. ARM/Embedded 크로스 컴파일

x86_64 호스트에서 aarch64 타겟 Rust 빌드를 검증하는 크로스 컴파일 환경임.

핵심 포인트: aarch64 · Jetson/Embedded · SSH build node

Docker Compose 예제

```
services:
  cross-build-server:
    image: ghcr.io/cross-rs/aarch64-unknown-linux-gnu:edge
    environment:
      TARGET: aarch64-unknown-linux-gnu
    ports: ["2226:22"]
    volumes: [./embedded_rust:/workspace]
    working_dir: /workspace
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server &&
      echo 'root:${CROSS_SSH_PASSWORD}' | chpasswd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config
&&
      mkdir -p /var/run/sshd &&
      /usr/sbin/sshd -D"
    networks: [cross-net]

networks:
  cross-net:
    driver: bridge
```

yaml

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

bash

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

섹션 14. Traefik API Gateway

Traefik이 Docker labels를 읽어 `/api/v1/user`는 Kotlin, `/api/v1/compute`는 Rust로 라우팅함.

핵심 포인트: Reverse proxy · path routing · Docker socket read-only

Docker Compose 예제

```
services:
  traefik:
    image: traefik:v3.0
    command:
      - --api.insecure=true
      - --providers.docker=true
      - --entrypoints.web.address=:80
    ports:
      - "80:80"
      - "8082:8080"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
    networks: [edge-net]

  kotlin-service:
    image: eclipse-temurin:21-jre-alpine
    volumes: [./kotlin_jar:/app]
    command: java -jar /app/app.jar
    labels:
      - traefik.enable=true
      - traefik.http.routers.kotlin.rule=PathPrefix(`/api/v1/user`)
      - traefik.http.services.kotlin.loadbalancer.server.port=8080
    networks: [edge-net]

  rust-service:
    image: rust:1.80-slim
    volumes: [./rust_bin:/app]
    command: /app/service_binary
    labels:
      - traefik.enable=true
      - traefik.http.routers.rust.rule=PathPrefix(`/api/v1/compute`)
      - traefik.http.services.rust.loadbalancer.server.port=9000
    networks: [edge-net]

networks:
  edge-net:
    driver: bridge
```

yaml

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

bash

섹션 15. Hardened 루트리스 개발

Non-root, cap_drop, no-new-privileges, resource limit을 사용해 보안 제약 하에서 Rust 테스트를 수행함.

핵심 포인트: Non-root · cap_drop ALL · resource limits

Docker Compose 예제

```
services:
  hardened-mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${HARDENED_DB_PASSWORD}
      MYSQL_DATABASE: secure_zone
    mem_limit: 1024m
    cpus: 1.5
    networks: [hardened-net]

  hardened-rust:
    image: rust:1.80-slim
    user: "1001:1001"
    working_dir: /home/appuser/src
    volumes: [./secure_code:/home/appuser/src]
    command: cargo test
    cap_drop: [ALL]
    security_opt:
      - no-new-privileges:true
    mem_limit: 512m
    cpus: 1.0
    networks: [hardened-net]

networks:
  hardened-net:
    driver: bridge
```

yaml

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

bash

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 16. 멀티 테넌트 격리 DB

Tenant A/B의 DB를 별도 컨테이너로 분리하고 Kotlin에서 tenant별 datasource routing을 검증함.

핵심 포인트: Tenant isolation · dynamic datasource · 2 DB

Docker Compose 예제

```
services:
  mariadb-tenant-a:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${TENANT_A_PASSWORD}
      MYSQL_DATABASE: tenant_a_db
    networks: [tenant-net]

  mariadb-tenant-b:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${TENANT_B_PASSWORD}
      MYSQL_DATABASE: tenant_b_db
    networks: [tenant-net]

  kotlin-router:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /app
    volumes: [./multitenant-src:/app]
    environment:
      TENANT_A_URL: jdbc:mariadb://mariadb-tenant-a:3306/tenant_a_db
      TENANT_B_URL: jdbc:mariadb://mariadb-tenant-b:3306/tenant_b_db
    command: ./gradlew bootRun
    ports: ["8080:8080"]
    networks: [tenant-net]

networks:
  tenant-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 17. 분산 작업 큐 & 배치 워커

Kotlin producer가 Redis에 작업을 발행하고 Rust consumer가 처리 후 MariaDB에 저장하는 비동기 배치 구조임.

핵심 포인트: Redis queue · Kotlin producer · Rust worker

Docker Compose 예제

```
services:
  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${BATCH_DB_PASSWORD}
      MYSQL_DATABASE: batch_store
    networks: [job-net]

  redis-queue:
    image: redis:7-alpine
    networks: [job-net]

  kotlin-job-producer:
    image: eclipse-temurin:21-jre-alpine
    volumes: [./producer_jar:/app]
    environment:
      REDIS_HOST: redis-queue
    command: java -jar /app/producer.jar
    networks: [job-net]

  rust-job-consumer:
    image: rust:1.80-bullseye
    working_dir: /worker
    volumes: [./rust_worker:/worker]
    environment:
      REDIS_URL: redis://redis-queue:6379
      DATABASE_URL: mysql://root:${BATCH_DB_PASSWORD}@mariadb:3306/batch_store
    command: cargo run --bin queue_worker
    networks: [job-net]

networks:
  job-net:
    driver: bridge
```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 18. 고성능 실시간 WebSocket

Rust WebSocket 서버와 Kotlin client simulator를 연결하고 세션 메타데이터를 MariaDB에 저장함.

핵심 포인트: WebSocket · Tokio · client simulator

Docker Compose 예제

```
services:
  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${WS_DB_PASSWORD}
      MYSQL_DATABASE: ws_sessions
    networks: [ws-net]

  rust-ws-server:
    image: rust:1.80-bullseye
    working_dir: /ws
    volumes: [./rust_ws:/ws]
    environment:
      DATABASE_URL: mysql://root:${WS_DB_PASSWORD}@mariadb:3306/ws_sessions
    command: cargo run --release
    ports: ["9001:9001"]
    networks: [ws-net]

  kotlin-client-sim:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /sim
    volumes: [./client_simulator:/sim]
    environment:
      WS_TARGET: ws://rust-ws-server:9001
    command: ./gradlew run
    depends_on: [rust-ws-server]
    networks: [ws-net]

networks:
  ws-net:
    driver: bridge
```

yaml

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

bash

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 19. 클라우드 IDE / Code-Server + Codex

브라우저 IDE, Codex CLI, JDK, Rust를 하나의 개발 컨테이너에 넣고 MariaDB를 별도 서비스로 연결하는 구성임.

핵심 포인트: Browser IDE · Codex · Rust/JDK · MariaDB

Docker Compose 예제

```
services:
  code-server-sandbox:
    image: codercom/code-server:latest
    ports: ["8443:8080"]
    environment:
      PASSWORD: ${IDE_PASSWORD}
      OPENAI_API_KEY: ${OPENAI_API_KEY}
    volumes:
      - ./ide_workspace:/home/coder/project
      - ide_config:/home/coder/.config
    entrypoint: >
      bash -c "sudo apt-get update &&
      sudo apt-get install -y default-jdk build-essential curl nodejs npm &&
      curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs |
      sh -s -- -y &&
      sudo npm install -g @openai/codex &&
      /usr/bin/entrypoint.sh --bind-addr 0.0.0.0:8080
      /home/coder/project"
    networks: [ide-net]

  mariadb:
    image: mariadb:11.2
    environment:
      MYSQL_ROOT_PASSWORD: ${IDE_DB_PASSWORD}
      MYSQL_DATABASE: dev_db
    ports: ["3310:3306"]
    networks: [ide-net]

volumes:
  ide_config:

networks:
  ide-net:
    driver: bridge
```

yaml

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100
```

bash

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

섹션 20. 로컬 프로덕션 모사 풀스택

MariaDB, Redis, 관리 SSH/Codex, Spring Boot, Rust worker, Prometheus를 통합하여 운영과 유사한 의존관계를 로컬에서 검증함.

핵심 포인트: 운영 모사 · Prometheus · Redis · Spring/Rust · 관리 SSH/Codex

Docker Compose 예제

yaml

```
services:
  mariadb:
    image: mariadb:11.2
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: ${PROD_DB_ROOT_PASSWORD}
      MYSQL_DATABASE: prod_cluster
      MYSQL_USER: service_app
      MYSQL_PASSWORD: ${PROD_DB_APP_PASSWORD}
    volumes: [prod_db_data:/var/lib/mysql]
    ports: ["3306:3306"]
    networks: [full-cluster]

  redis:
    image: redis:7.2-alpine
    networks: [full-cluster]

  ssh-management-node:
    image: ubuntu:24.04
    command: >
      bash -c "apt-get update &&
      apt-get install -y openssh-server git curl mariadb-client nodejs npm &&
      mkdir -p /var/run/sshd &&
      echo 'root:${MGMT_SSH_PASSWORD}' | chpasswd &&
      sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config
&&
      npm install -g @openai/codex &&
      /usr/sbin/sshd -D"
    ports: ["2227:22"]
    environment:
      OPENAI_API_KEY: ${OPENAI_API_KEY}
    networks: [full-cluster]

  springboot-core:
    image: eclipse-temurin:21-jdk-jammy
    working_dir: /workspace
    volumes: [./springboot_core:/workspace]
    environment:
      SPRING_DATASOURCE_URL: jdbc:mariadb://mariadb:3306/prod_cluster
      SPRING_DATASOURCE_USERNAME: service_app
      SPRING_DATASOURCE_PASSWORD: ${PROD_DB_APP_PASSWORD}
      SPRING_DATA_REDIS_HOST: redis
      SPRING_DATA_REDIS_PORT: "6379"
    command: ./gradlew bootRun
    ports: ["8080:8080"]
    depends_on: [mariadb, redis]
    networks: [full-cluster]

  rust-analyzer-worker:
    image: rust:1.80-bullseye
    working_dir: /analyzer
    volumes: [./rust_analyzer:/analyzer]
    environment:
      DATABASE_URL: mysql://service_app:${PROD_DB_APP_PASSWORD}@mariadb:3306/prod_cluster
      REDIS_URL: redis://redis:6379
    command: cargo run --release
```

```

    depends_on: [mariadb, redis]
    networks: [full-cluster]

    prometheus:
      image: prom/prometheus:v2.51.0
      ports: ["9090:9090"]
      volumes:
        - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      networks: [full-cluster]

volumes:
  prod_db_data:

networks:
  full-cluster:
    driver: bridge

```

구동 및 확인

해당 예제를 `docker-compose.yml`로 저장한 후 다음 순서로 검증함.

```

docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail=100

```

bash

SSH가 포함된 경우 포트는 해당 섹션의 `ports` 값을 사용하여 접속함.

DB가 포함된 경우 애플리케이션 컨테이너에서는 호스트 포트가 아니라 Compose 서비스명과 내부 3306 포트를 사용함.

21. 공통 .env 파일 작성

문서에 포함된 `\${...}` 값은 실제 비밀번호나 API 키를 Compose 파일에 직접 기록하지 않도록 분리한 것임. 아래는 예시이며 실제 값으로 교체하고 `.env`는 Git에 커밋하지 않음.

```
OPENAI_API_KEY=YOUR_OPENAI_API_KEY
```

env

```
DB_ROOT_PASSWORD=CHANGE_ME
DB_APP_PASSWORD=CHANGE_ME
SSH_DEV_PASSWORD=CHANGE_ME
SSH_RUST_PASSWORD=CHANGE_ME
MASTER_ROOT_PASSWORD=CHANGE_ME
SLAVE_ROOT_PASSWORD=CHANGE_ME
REPL_PASSWORD=CHANGE_ME
PERF_DB_PASSWORD=CHANGE_ME
SSH_LLM_PASSWORD=CHANGE_ME
POLYGLOT_DB_PASSWORD=CHANGE_ME
BASTION_PASSWORD=CHANGE_ME
INTERNAL_DB_PASSWORD=CHANGE_ME
TS_DB_PASSWORD=CHANGE_ME
TUI_DB_PASSWORD=CHANGE_ME
TUI_SSH_PASSWORD=CHANGE_ME
FORENSICS_DB_PASSWORD=CHANGE_ME
EVENT_DB_PASSWORD=CHANGE_ME
CROSS_SSH_PASSWORD=CHANGE_ME
HARDENED_DB_PASSWORD=CHANGE_ME
TENANT_A_PASSWORD=CHANGE_ME
TENANT_B_PASSWORD=CHANGE_ME
BATCH_DB_PASSWORD=CHANGE_ME
WS_DB_PASSWORD=CHANGE_ME
IDE_PASSWORD=CHANGE_ME
IDE_DB_PASSWORD=CHANGE_ME
PROD_DB_ROOT_PASSWORD=CHANGE_ME
PROD_DB_APP_PASSWORD=CHANGE_ME
MGMT_SSH_PASSWORD=CHANGE_ME
```

22. 공통 서버 구동 및 검증 절차

22.1 Compose 문법 검사

```
docker compose config
```

bash

YAML 들여쓰기, 변수 치환, Compose schema 문제를 먼저 확인함.

22.2 백그라운드 구동

```
docker compose up -d --build
docker compose ps
```

bash

22.3 로그 확인

```
docker compose logs -f
docker compose logs --tail=200 mariadb
docker compose logs --tail=200 kotlin-springboot
```

bash

22.4 SSH 접속

```
ssh root@localhost -p 2222
```

bash

실제 포트는 각 섹션의 Compose 설정을 확인함. 로컬 실습이 아니라면 root password SSH 대신 key 인증과 일반 사용자를 권장함.

22.5 Codex CLI 확인

```
docker exec -it sbx-ssh-codex codex --version
docker exec -it sbx-ssh-codex codex
```

bash

Codex CLI의 설치/인증 방식은 버전에 따라 변경될 수 있으므로 실제 구축 시 현재 공식 OpenAI Codex 문서를 확인함.

22.6 MariaDB 확인

```
docker exec -it sbx-mariadb mariadb -u devuser -p -e "SHOW DATABASES;"
docker compose exec mariadb mariadb-admin ping -h localhost
```

bash

22.7 Spring Boot 확인

```
curl -v http://localhost:8080/
docker compose logs -f kotlin-springboot
```

bash

22.8 Rust 확인

```
docker compose exec rust-worker rustc --version
docker compose exec rust-worker cargo --version
docker compose logs -f rust-worker
```

bash

22.9 네트워크 확인

```
docker network ls
docker compose exec kotlin-springboot getent hosts mariadb
docker compose exec rust-worker getent hosts mariadb
```

bash

22.10 종료

```
# 컨테이너/네트워크 제거, named volume 유지
docker compose down

# named volume까지 제거 - DB 데이터 삭제 주의
docker compose down -v
```

bash

23. 개발/격리 검증 체크리스트

- 외부에 꼭 필요한 포트만 publish했는가?
- DB 접속 시 컨테이너 간에는 서비스명:내부포트를 사용하는가?
- 비밀번호/API 키가 Compose 파일과 Git에 하드코딩되어 있지 않은가?
- DB가 영구 보존되어야 하면 named volume이 있는가?
- CI용 DB는 tmpfs 등 휘발성 저장이 더 적합한가?
- 증거/원본 파일은 read-only mount가 필요한가?
- Bastion/secure backend에는 internal network가 적절한가?
- Rust/Kotlin 빌드 cache를 volume으로 분리할 필요가 있는가?
- healthcheck가 필요한 핵심 의존 서비스에 설정되어 있는가?

- 실제 운영 전에는 latest 태그 대신 검증된 고정 버전을 사용하는가?

24. 권장 사용 구분

목적	권장 섹션
일반 Kotlin/Rust 풀스택 개발	1
SSH 원격 Rust 개발	2
Spring 다중 DB/복제 테스트	3
Rust 성능/Redis	4, 9
로컬 LLM	5
Kotlin-Rust RPC	6
CI 테스트	7
네트워크 격리	8, 15
포렌식/파일 분석	11
Kafka 이벤트 처리	12
Jetson/aarch64 빌드	13
Gateway/Reverse Proxy	14
멀티테넌트	16
비동기 Job Queue	17
WebSocket	18
웹 IDE	19
통합 운영 모사	20

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.